Lesson 3 — Express Middleware and Error Handling

Node.js/Express

Game Plan

- Warm-Up
- Express request flow
- Middleware and `next()`
- Middleware order
- Built-in, custom, and third-party middleware
- Not-found and error handlers
- Status codes and error responses
- Debugging Express
- Assignment Preview

Warm-Up (5 min)

In chat or out loud:

1. What did you build for Assignment 2?
2. What still feels unclear about Express?

Lesson 2 Recap

Students already saw:

- Basic Express route handlers
- `req` and `res`
- `express.json()`
- Routes and controllers
- Postman for POST requests

Today:

> What happens around route handlers?

Express Request Flow

```
request -> middleware -> route handler -> response
```

Middleware can:

- Prepare the request
- Check the request
- Modify `req` or `res`
- Stop with a response
- Pass control with `next()`

What Is Middleware?

```
function logger(req, res, next) {
  console.log(`${req.method} ${req.path}`);
  next();
}

app.use(logger);
```

Middleware receives:

- `req`
- `res`
- `next`

What Does `next()` Mean?

Think of `next()` as:

> "I am done with my part. Go to the next matching function."

Every middleware must:

1. Send a response
2. Call `next()`
3. Call `next(error)` or throw

Otherwise the request hangs.

Middleware vs Route Handler

	Middleware	Route Handler
Registered with	`app.use()`	`app.get()`, `app.post()`
Typical job	prepare/check/log/modify	send main route response
Usually calls	`next()`	`res.json()` / `res.send()`

Route handlers are usually the endpoint of the chain.

Order Matters

```
app.use(logger);
app.use(express.json());

app.get("/tasks", getTasks);
app.post("/tasks", createTask);

app.use(notFound);
app.use(errorHandler);
```

Express checks these from top to bottom.

Path Matching

Code	Matches	Does not match
`app.use("/api", logger)`	`/api/tasks`	`/tasks`
`app.get("/api", handler)`	`GET /api`	`GET /api/tasks`

`app.use()` can match a prefix.

Route methods match a specific method and path.

Built-In Middleware

Most important for this course:

```
app.use(express.json());
```

It parses JSON request bodies and puts the object on:

```
req.body
```

Order reminder:
`express.json()` must come before routes that read `req.body`.

Custom Middleware

Request logger:

```
function requestLogger(req, res, next) {
  console.log(`${req.method} ${req.path}`);
  next();
}
```

Request metadata:

```
function addRequestTime(req, res, next) {
  req.requestTime = new Date().toISOString();
  next();
}
```

Content-Type Checker

Headers are metadata.

`Content-Type: application/json` means:

> "The request body should be JSON."

```
if (methodsWithBody.includes(req.method) && !req.is("application/json")) {
  return res.status(400).json({
    message: "Content-Type must be application/json.",
  });
}
next();
```

Why `return`?

```
return res.status(400).json({
  message: "Content-Type must be application/json.",
});
```

The `return` stops the middleware.

Without it, the function could keep going and call `next()` after a response was already sent.

Third-Party Middleware

Installed from npm:

```
npm install morgan
```

Used like middleware:

```
const morgan = require("morgan");

app.use(morgan("dev"));
```

If not installed: `Cannot find module`.

Common Third-Party Middleware

- `morgan` : logs requests
- `cors` : configures cross-origin requests
- `cookie-parser` : parses cookies into `req.cookies`
- `helmet` : sets helpful security-related headers
- `compression` : compresses responses

Ordering still matters.

Example: `morgan` usually goes near the top.

Modifying `req`

The same `req` object moves through the chain.

```
app.use((req, res, next) => {
  req.requestTime = new Date().toISOString();
  next();
});

app.get("/debug", (req, res) => {
  res.json({ requestTime: req.requestTime });
});
```

Modifying `res`

Middleware can set response headers:

```
app.use((req, res, next) => {
  res.setHeader("X-App-Name", "Node Homework");
  next();
});
```

It can also listen for the response finishing:

```
res.on("finish", () => {
  console.log(res.statusCode);
});
```

Not-Found Middleware

Not-found means:

> No route matched this request.

```
app.use((req, res) => {
  res.status(404).json({
    message: `No route found for ${req.method} ${req.path}`,
  });
});
```

Place after all real routes.

Error Handling Middleware

Error handlers have **four parameters**:

```
app.use((err, req, res, next) => {
  console.error(err);
  res.status(500).json({
    message: "Internal server error.",
  });
});
```

Express recognizes this signature:

```
(err, req, res, next)
```

Not Found vs Error Handler

Situation	Handler
No route matched	Not-found middleware
Something broke while processing	Error middleware

Examples:

- `/not-real` -> 404 not found
- database throws -> error handler

Async Error Pattern

```
app.get("/tasks/:id", async (req, res, next) => {
  try {
    const task = await getTaskById(req.params.id);
    if (!task) return res.status(404).json({ message: "Task not found." });
    res.status(200).json({ task });
  } catch (error) {
    next(error);
  }
});
```

For callbacks: use `next(error)`, do not throw inside the callback.

Status Codes

Common API codes:

- `200 OK` : success with data
- `201 Created` : new resource created
- `204 No Content` : success with no body
- `400 Bad Request` : bad request data
- `401 Unauthorized` : not logged in / invalid auth
- `403 Forbidden` : logged in but not allowed
- `404 Not Found` : route or resource missing
- `500 Internal Server Error` : unexpected server failure

Don't Use 500 For Everything

Use `4xx` when the client caused the problem:

```
res.status(400).json({ message: "Title is required." });
res.status(404).json({ message: "Task not found." });
```

Use `500` when the server failed unexpectedly:

```
res.status(500).json({ message: "Internal server error." });
```

Error Response Shape

Simple and consistent:

```
{
  message: "Title is required."
}
```

Larger apps may add:

```
{
  message: "Validation failed.",
  errors: [{ field: "email", message: "Email must be valid." }]
}
```

Do not send stack traces to clients.

Complete Mini App

Core order:

```
app.use(logger);
app.use(express.json());

app.get("/", home);
app.post("/echo", echo);
app.get("/problem", (req, res, next) => next(new Error("Boom")));

app.use(notFound);
app.use(errorHandler);
```

This is the pattern to keep using.

We Do — Trace a Request

Request:

```
POST /echo
Content-Type: application/json

{ "message": "Hello" }
```

Trace:

1. logger
2. `express.json()`
3. `POST /echo`
4. response

What is `req.body`?

We Do — Debug This

What's wrong?

```
app.post("/users", (req, res, next) => {
  res.status(201).json({ user: req.body });
  next();
});
```

Basic Debugging

Check:

1. Did the request reach the server?
2. Did method/path match?
3. Did middleware call `next()`?
4. Did a middleware send a response early?
5. Is `express.json()` before routes?
6. Did the error handler run?

Use Postman or browser network tab to inspect status and body.

Assignment Preview

Assignment 3 has three parts:

1. Assignment 3A: extend the project app
    - user register/logon/logoff
    - controllers and routers
2. Assignment 3B: work in week 3 folder
    - JSON body parsing
    - static files
    - request IDs with `crypto.randomUUID()`
    - logging
    - 404 handler

Assignment Commands

Core app work:

```
npm run tdd assignment3a
```

Middleware/debugging work:

```
npm run week3
npm run tdd assignment3b
npm run tdd assignment3c # optional advanced
```

Use Postman to test routes manually.

Wrap-Up

In chat:

1. What is `next()` for?
2. Why does middleware order matter?
3. What is the difference between 404 and 500?
4. Why does the error handler have four parameters?

Confidence Check

On a scale of 1-5:

How comfortable do you feel building middleware and error handlers?

Resources

- https://expressjs.com/en/guide/writing-middleware.html
- https://expressjs.com/en/guide/error-handling.html
- https://expressjs.com/en/resources/middleware.html
- Ask questions in Slack

Closing

This week:

Middleware, request flow, status codes, error handling, and debugging.

Next week:

Security middleware, validation, and password hashing.

See you then!